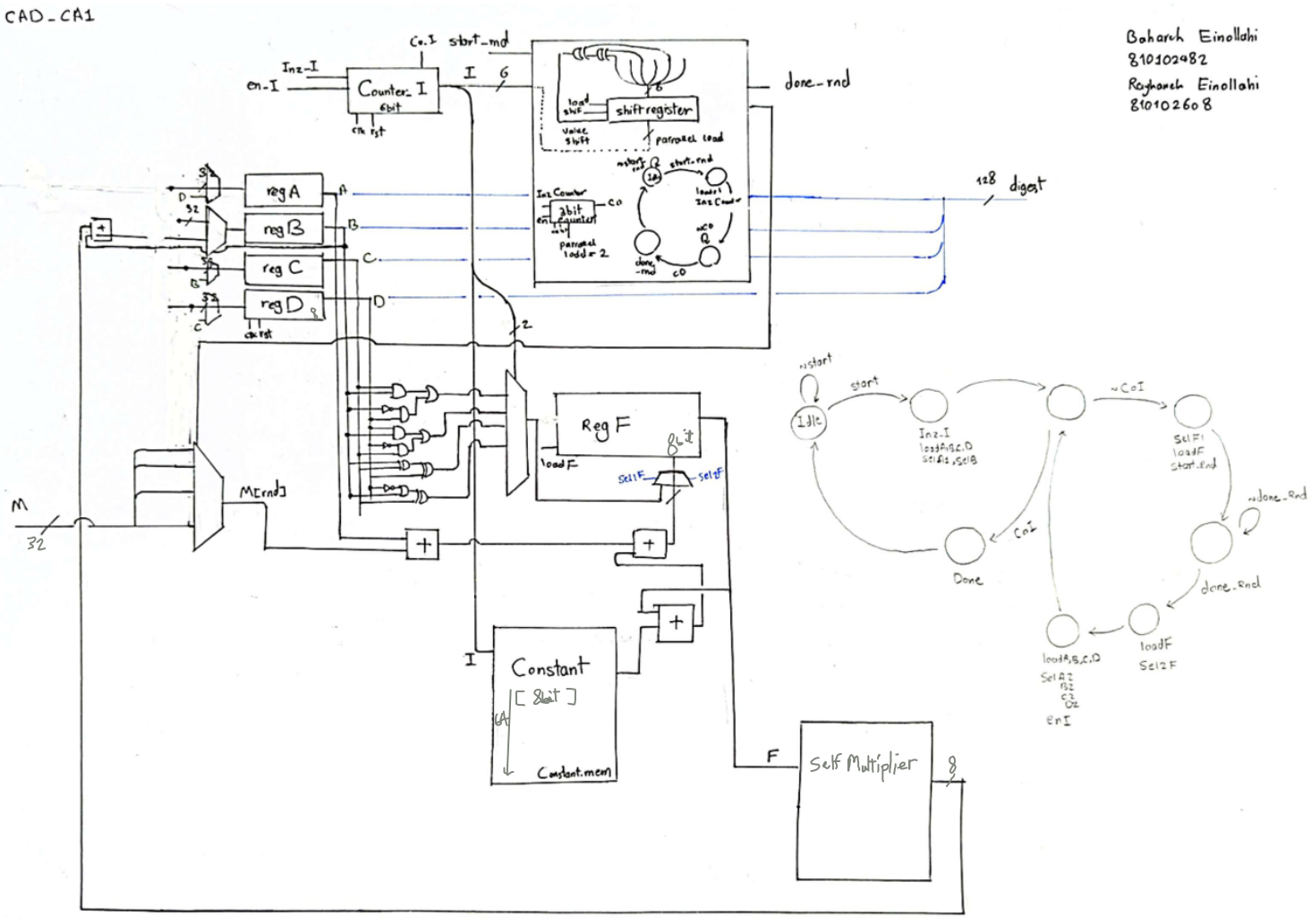
CA3

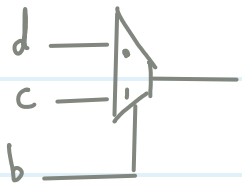
Bahareh Einollahi 810102482

Rayhaneh Einollahi 810102608

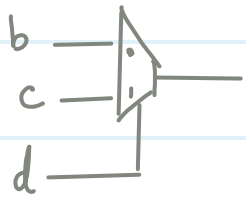


```
assign input_f1 =  (out_counter_i[5:4] == 2'b00) ? ((out_b & out_c) | ((~out_b) & out_d)) :
                   (out_counter_i[5:4] == 2'b01) ? ((out_d & out_b) | ((~out_d) & out_c)) :
                   (out_counter_i[5:4] == 2'b10) ? (out_b ^ out_c ^ out_d ) :
                                                   (out_c ^ (out_b | (~out_d)));
```

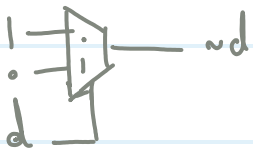
→ $(b \,\&\, c) \mid (\sim b \,\&\, d)$



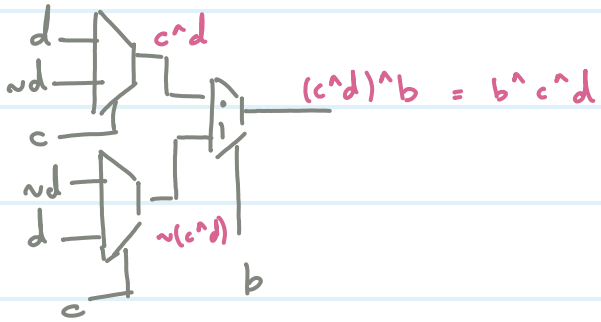
→ $(d \,\&\, b) \mid (\sim d \,\&\, c)$



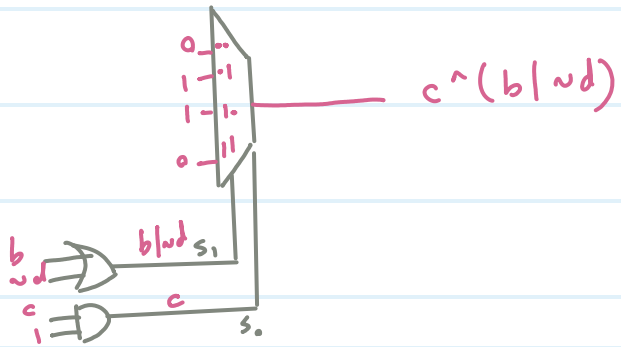
→ $\sim d$



→ $b \wedge c \wedge d$



→ $c \wedge (b \mid \sim d)$



```
module build_f1(
    input a,
    input b,
    input c,
    input d,
    input [1:0]sel,
    output out
);


    wire [3:0] res;
    wire out_res0_res1;
    wire d_inv;
    logic_not not1(d, d_inv);
    //(b & c) | (~b & d)
    //(d & b) | (~d & c)
    c1 c12(.A0(d) , .A1(c) , .SA(b) , .B0(c) , .B1(b) , .SB(d) , .S0(sel[0]) , .S1(sel[0]),
.f(out_res0_res1)) ;
    //b ^ c ^ d
    c1 c13(.A0(d) , .A1(d_inv) , .SA(c) , .B0(d_inv) , .B1(d) , .SB(c) , .S0(b) , .S1(b),.f(res[2])) ;
    //c ^ (b | ~d)
    c2 c24(.D00(1'b0), .D01(1'b1), .D10(1'b1), .D11(1'b0) , .A1(b) , .B1(d_inv) , .A0(c) , .B0(c) ,
.out(res[3]));

    c2 c25(.D00(out_res0_res1), .D01(out_res0_res1), .D10(res[2]), .D11(res[3]) , .A1(sel[1]) ,
.B1(sel[1]) , .A0(sel[0]) , .B0(sel[0]) , .out(out));

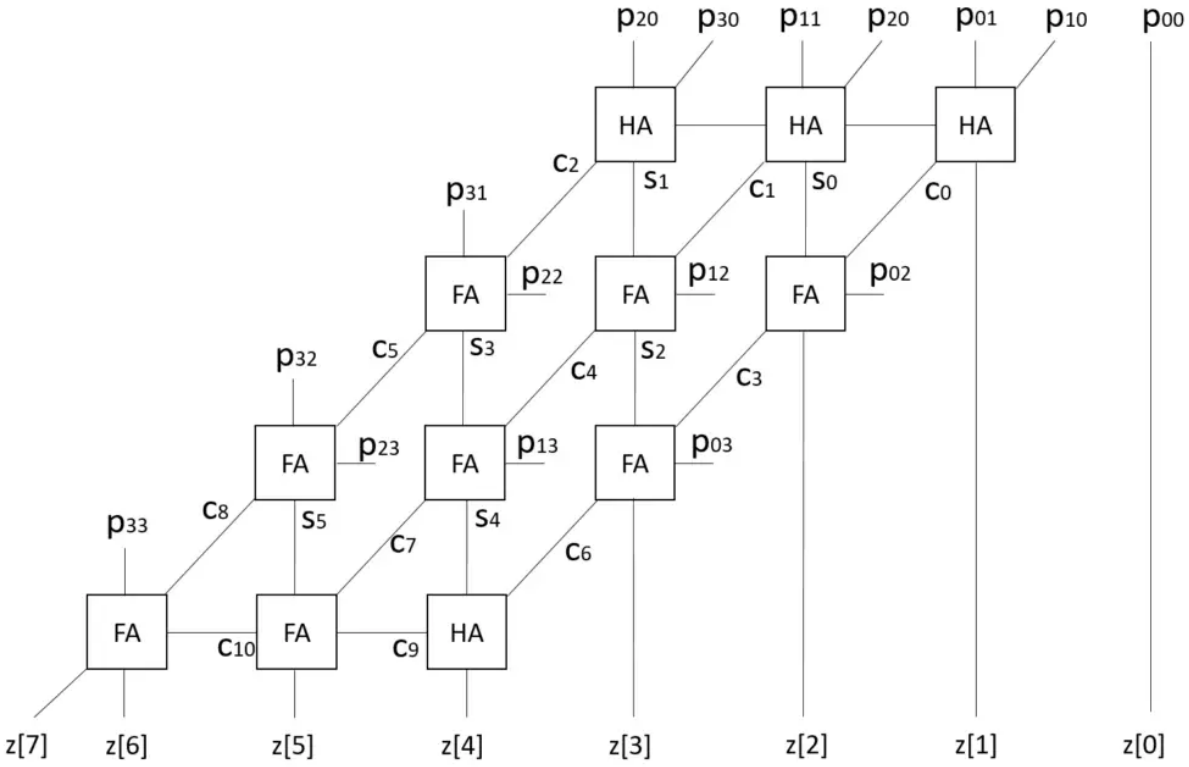
endmodule


module parallel_build_f1 (
    input  [7:0] a,
    input  [7:0] b,
    input  [7:0] c,
    input  [7:0] d,
    input  [1:0] sel,
    output [7:0] out
);

genvar i;
generate
    for (i = 0; i < 8; i = i + 1) begin : f1_gen
        build_f1 f1_unit (
            .a(a[i]),
            .b(b[i]),
            .c(c[i]),
            .d(d[i]),
            .sel(sel),
            .out(out[i])
        );
    end
endgenerate

endmodule
```

Matrix Multiplier



Array Multiplier

```
//-------------------------------------------------------------
// number of gates:                                        328
//-------------------------------------------------------------

module multiplier(
    input [7:0] f,
    output [7:0] multiplier_out
);
    wire [3:0] a;
    wire [3:0] b;
    assign a = f[7:4];
    assign b = f[3:0];

    wire pp0_0, pp0_1, pp0_2, pp0_3;
    wire pp1_0, pp1_1, pp1_2, pp1_3;
    wire pp2_0, pp2_1, pp2_2, pp2_3;
    wire pp3_0, pp3_1, pp3_2, pp3_3;

    logic_and2 pa00 (.a(a[0]), .b(b[0]), .out(pp0_0));
    logic_and2 pa01 (.a(a[1]), .b(b[0]), .out(pp0_1));
    logic_and2 pa02 (.a(a[2]), .b(b[0]), .out(pp0_2));
    logic_and2 pa03 (.a(a[3]), .b(b[0]), .out(pp0_3));

    logic_and2 pa10 (.a(a[0]), .b(b[1]), .out(pp1_0));
    logic_and2 pa11 (.a(a[1]), .b(b[1]), .out(pp1_1));
    logic_and2 pa12 (.a(a[2]), .b(b[1]), .out(pp1_2));
    logic_and2 pa13 (.a(a[3]), .b(b[1]), .out(pp1_3));

    logic_and2 pa20 (.a(a[0]), .b(b[2]), .out(pp2_0));
    logic_and2 pa21 (.a(a[1]), .b(b[2]), .out(pp2_1));
    logic_and2 pa22 (.a(a[2]), .b(b[2]), .out(pp2_2));
    logic_and2 pa23 (.a(a[3]), .b(b[2]), .out(pp2_3));

    logic_and2 pa30 (.a(a[0]), .b(b[3]), .out(pp3_0));
    logic_and2 pa31 (.a(a[1]), .b(b[3]), .out(pp3_1));
    logic_and2 pa32 (.a(a[2]), .b(b[3]), .out(pp3_2));
    logic_and2 pa33 (.a(a[3]), .b(b[3]), .out(pp3_3));

    wire s1_0, c1_0, s1_1, c1_1, s1_2, c1_2, s1_3, c1_3;
    wire s2_0, c2_0, s2_1, c2_1, s2_2, c2_2, s2_3, c2_3;
    wire s3_0, c3_0, s3_1, c3_1, s3_2, c3_2, s3_3, c3_3;

    assign multiplier_out[0] = pp0_0;

    half_adder ha1_0 (.a(pp0_1), .b(pp1_0), .sum(s1_0), .cout(c1_0));
    full_adder fa1_1 (.a(pp0_2), .b(pp1_1), .cin(c1_0), .sum(s1_1), .cout(c1_1));
    full_adder fa1_2 (.a(pp0_3), .b(pp1_2), .cin(c1_1), .sum(s1_2), .cout(c1_2));
    half_adder ha1_3 (.a(pp1_3), .b(c1_2), .sum(s1_3), .cout(c1_3));

    assign multiplier_out[1] = s1_0;

    half_adder ha2_0 (.a(s1_1), .b(pp2_0), .sum(s2_0), .cout(c2_0));
    full_adder fa2_1 (.a(s1_2), .b(pp2_1), .cin(c2_0), .sum(s2_1), .cout(c2_1));
    full_adder fa2_2 (.a(s1_3), .b(pp2_2), .cin(c2_1), .sum(s2_2), .cout(c2_2));
    full_adder fa2_3 (.a(c1_3), .b(pp2_3), .cin(c2_2), .sum(s2_3), .cout(c2_3));

    assign multiplier_out[2] = s2_0;

    half_adder ha3_0 (.a(s2_1), .b(pp3_0), .sum(s3_0), .cout(c3_0));
    full_adder fa3_1 (.a(s2_2), .b(pp3_1), .cin(c3_0), .sum(s3_1), .cout(c3_1));
    full_adder fa3_2 (.a(s2_3), .b(pp3_2), .cin(c3_1), .sum(s3_2), .cout(c3_2));
    full_adder fa3_3 (.a(c2_3), .b(pp3_3), .cin(c3_2), .sum(s3_3), .cout(c3_3));

    assign multiplier_out[3] = s3_0;
    assign multiplier_out[4] = s3_1;
    assign multiplier_out[5] = s3_2;
    assign multiplier_out[6] = s3_3;
    assign multiplier_out[7] = c3_3;

endmodule
```